DATABASE AND COMPUTER NETWORKS

CIS095-1

Assignment 1

Aspire Fitness – Database Design

Student Name: Kripa Parajuli

Student ID: 2510112344

Section: B (L4 SEM2)

Submission Date: 13 March, 2026

Table of Contents

Introduction 1

Scenario Description 2

Assumptions 3

Task 1: Entity Relationship Model 4

- Entities and Their Attributes 4
- Relationships Between Entities 4
- Entity Relationship Diagram 5

Task 2: Physical Database Design 7

- Skeleton Tables (Normalized Relations) 7
- Data Dictionary 7
  - Table 1: Member 7
  - Table 2: FitnessStaff 8
  - Table 3: Room 8
  - Table 4: Class 9
  - Table 5: FacilityBooking 9
  - Table 6: ClassBooking 10

Task 3: Implementation of the Database 11

- Part A – Create Tables 11
  - Create Table: Member 11
  - Create Table: FitnessStaff 11
  - Create Table: Room 12
  - Create Table: Class 12
  - Create Table: FacilityBooking 13
  - Create Table: ClassBooking 14
- Part B – Insert Data 14
  - Insert Data: Member 14
  - Insert Data: FitnessStaff 15
  - Insert Data: Room 15
  - Insert Data: Class 16
  - Insert Data: FacilityBooking 16

Insert Data: ClassBooking 17
Task 4: Query Design and Implementation 18
Part A – List of Queries and Their Purpose 18
Part B – SQL Queries Implemented 18
Query 1: Full Member List 18
Query 2: Current Week Facility Bookings 19
Query 3: Classes by a Specific Instructor 20
Query 4: Classes in a Specific Room 20
Query 5: Member Booking Summary 21
Query 6: Class Bookings Count per Member 22
Query 7: Most Active Members This Month 23
Query 8: Members Over Free Class Limit 24
Query 9: Room Usage Summary 25
Query 10: Members with Extra Charges 26
Discussion and Critical Reflection 27
Conclusion 27
References 28
Appendix 29

Table of Figures

Figure 1: Entity Relationship Diagram for Aspire Fitness 6
Figure 2: Member table created 11
Figure 3: FitnessStaff table created 12
Figure 4: Room table created 12
Figure 5: Class table created 13
Figure 6: FacilityBooking table created 13
Figure 7: ClassBooking table created 14
Figure 8: Member table with inserted data 14
Figure 9: FitnessStaff table with inserted data 15
Figure 10: Room table with inserted data 15
Figure 11: Class table with inserted data 16
Figure 12: FacilityBooking table with inserted data 16
Figure 13: ClassBooking table with inserted data 17
Figure 14: Query 1 output 19
Figure 15: Query 2 output 20
Figure 16: Query 3 output 20
Figure 17: Query 4 output 21
Figure 18: Query 5 output 22
Figure 19: Query 6 output 23
Figure 20: Query 7 output 24
Figure 21: Query 8 output 25
Figure 22: Query 9 output 26
Figure 23: Query 10 output 26

Abstract

This report presents the design and implementation of a relational database for Aspire Fitness, a local sports and fitness club. The database was designed to manage member bookings for both room facilities and fitness classes. The design process involved creating an Entity Relationship Model, normalizing the tables, and implementing the database using MySQL. Ten SQL queries were developed to demonstrate that the system meets the operational requirements of the club.

Introduction

For this assignment, I was given a task to design and build a database for a local sports club called Aspire Fitness. The goal was to follow the full database design process from understanding the business needs, building an Entity Relationship Model, normalizing the tables, and finally implementing everything in MySQL.

I was asked to complete four main tasks. First, identify the entities and relationships and produce an ERD. Second, create skeleton tables and a data dictionary. Third, write SQL to create the tables and insert sample data. Finally, design and implement queries to show the database supports the club's daily operations.

Since I had some experience with databases before this unit, I was familiar with primary keys, foreign keys, and basic SQL. However, this assignment took my understanding further by making me think about how to properly organize data and turn real business rules into a working database. Overall, I successfully completed all four tasks and the final database reflects the requirements of Aspire Fitness.

Scenario Description

Aspire Fitness is a local sports and fitness club with three bookable rooms: a Main Hall, a Yoga Hall, and a Small Dance Room. The club employs about 20 full-time staff including personal trainers, class instructors, and admin staff.

The database manages two main services. The first is room bookings where members can book one room per week for a maximum of two hours. Bookings cannot overlap with other bookings or scheduled classes. The second is class bookings where members can attend up to five classes per week for free. Each extra class above five costs Rs 500.

Every member gets a unique membership number and their personal details are stored in the database. Staff details are also recorded. Each member pays a weekly fee of Rs 1,000 plus Rs 500 for each extra class booked above five.

The following business rules apply:

- Members can only book one room per week for a maximum of two hours.
- Room bookings cannot overlap with other bookings or class times.
- Members get five free classes per week and pay Rs 500 for each extra class.
- Each class has a unique code, one instructor, and a capacity limit.
- Weekly charge is Rs 1,000 subscription plus any extra class charges.

Assumptions

When designing the database, I made some assumptions about things that were not clearly explained in the scenario. These assumptions helped me decide how to structure the tables and relationships:

1. A member must be registered in the system before making any room or class booking.
2. Each member can only make one room booking per week. Two or more bookings in the same week are not allowed.
3. Each room booking is for one room only and can last a maximum of two hours.
4. Each class has a fixed room, day, and time. The timetable only changes if the database is updated.
5. Each class is linked to only one instructor. If the instructor changes, the database must be updated.
6. Members can book as many classes as they want. The first five are free and each extra class costs Rs 500.
7. The medical conditions field is optional. Members with no health conditions can leave it empty.
8. Not all staff teach classes. Staff like managers and receptionists are still stored in the system but have no classes linked to them.
9. The weekly fee is not stored directly in the database. It is calculated by counting how many classes a member has booked that week.
10. The database does not automatically check for overlapping bookings. Staff are responsible for checking room availability before confirming a booking.

Task 1: Entity Relationship Model

An Entity Relationship Model (ERM) is a diagram used to plan what data a system needs to store and how different pieces of data are connected. Each box in the diagram represents an entity, which is something we want to store information about. The lines between the boxes show how the entities are related to each other.

Entities and Their Attributes

After reading the Aspire Fitness scenario, I identified six entities that the database needs to store information about:

Entity	Information Stored
Member	Stores each member's personal details including their unique membership number, name, address, date of birth, phone number, email, and any medical conditions.
FitnessStaff	Stores each staff member's unique staff number, name, job role, and contact phone number.
Room	Stores each room's unique room ID, room name, and maximum capacity.
Class	Stores each class's unique class code, class name, assigned instructor, room, day, start time, and maximum number of members who can attend.
FacilityBooking	Records each room booking including the booking ID, member, room, date, start and end time, and planned activity.
ClassBooking	Records each class booking including the booking ID, member, class code, booking date, and whether an extra charge applies.

Relationships Between Entities

Once I had all the entities, I worked out how they connect to each other. These connections are called relationships. Below I have described each relationship and its cardinality, which means how many records on each side can be linked together.

MEMBER ↔ FacilityBooking:

- A member can make many facility bookings over time.
- But each facility booking always belongs to just one member.
- Cardinality: One-to-Many (1:M)

MEMBER ↔ ClassBooking:

- A member can sign up for many class bookings.

- But each class booking is always linked to only one member.
- Cardinality: One-to-Many (1:M)

ROOM ↔ FacilityBooking:

- A room can appear in many facility bookings because different members can book the same room on different days.
- But each individual booking is always for one specific room.
- Cardinality: One-to-Many (1:M)

ROOM ↔ Class:

- A room can host many different classes throughout the week.
- But each class is always assigned to just one specific room.
- Cardinality: One-to-Many (1:M)

FitnessStaff ↔ Class:

- A staff member who is an instructor can teach many classes.
- But each class has only one instructor assigned to it.
- Cardinality: One-to-Many (1:M)

Class ↔ ClassBooking:

- A class can have many bookings from different members.
- But each individual class booking is always for one specific class.
- Cardinality: One-to-Many (1:M)

Entity Relationship Diagram

The ERD below was drawn using Visual Paradigm. It shows all six entities, their attributes, and how they are connected. The symbols at the ends of the lines show the type of relationship between each entity.

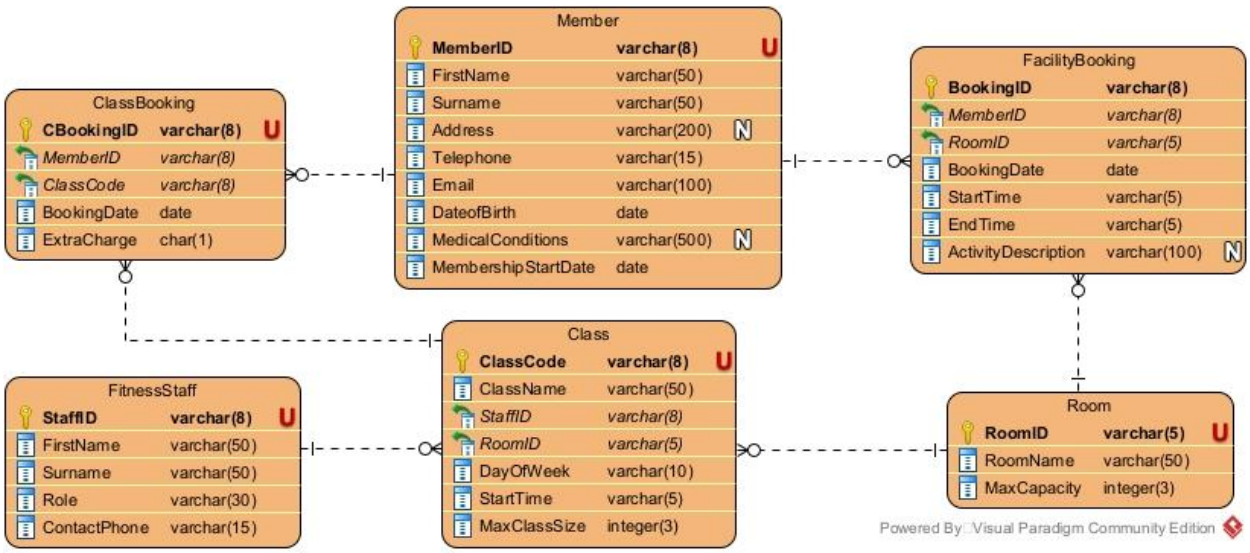

Figure 1: Entity Relationship Diagram for Aspire Fitness

Task 2: Physical Database Design

This section shows how the database was planned and structured before writing any code. I used MySQL for this project as it is free and widely used in both education and the workplace. I used MySQL Workbench to write and run SQL code and view the results.

Skeleton Tables (Normalized Relations)

Skeleton tables are a simple way to list all the tables in the database and what columns each one has. The underlined column is the Primary Key, which means it uniquely identifies each row. Columns marked with * are Foreign Keys, which means they link to another table. The tables below are organized so that there is no repeated or unnecessary data stored anywhere.

- Member (MemberID, FirstName, Surname, Address, Telephone, Email, DateOfBirth, MedicalConditions, MembershipStartDate)
- FitnessStaff (StaffID, FirstName, Surname, Role, ContactPhone)
- Room (RoomID, RoomName, MaxCapacity)
- Class (ClassCode, ClassName, StaffID*, RoomID*, DayOfWeek, StartTime, MaxClassSize)
- FacilityBooking (BookingID, MemberID*, RoomID*, BookingDate, StartTime, EndTime, ActivityDescription)
- ClassBooking (CBookingID, MemberID*, ClassCode*, BookingDate, ExtraCharge)

The tables in this database have been normalized to Third Normal Form (3NF). Each table has a single primary key and all columns depend only on that primary key. There are no repeating groups, no partial dependencies, and no transitive dependencies. For example, instructor details are stored in the FitnessStaff table rather than inside the Class or ClassBooking table, which avoids data repetition.

Data Dictionary

The following tables provide the data dictionary for each entity in the database design. Each table lists the field names, data types, and other important details about every column.

Table 1: Member

Field Name	Data Type	Length	Index	Null?	Validation	Description
MemberID	VARCHAR	8	PK	No		Unique identifier for each member
FirstName	VARCHAR	50		No		Member's first name

Surname	VARCHAR	50		No		Member's surname
Address	VARCHAR	200		No		Full postal address
Telephone	VARCHAR	15		No		Contact phone number
Email	VARCHAR	100		No	Must contain @	Email address
DateOfBirth	DATE			No	Must be past date	Member's date of birth
MedicalConditions	VARCHAR	500		Yes		Any declared medical conditions (optional)
MembershipStartDate	DATE			No		Date membership began

Table 2: FitnessStaff

Field Name	Data Type	Length	Index	Null?	Validation	Description
StaffID	VARCHAR	8	PK	No		Unique identifier for each staff member
FirstName	VARCHAR	50		No		Staff first name
Surname	VARCHAR	50		No		Staff surname
Role	VARCHAR	30		No	Personal Trainer, Class Instructor, Gym Staff, Manager, Administration	Staff role
ContactPhone	VARCHAR	15		No		Staff contact number

Table 3: Room

Field Name	Data Type	Length	Index	Null?	Validation	Description
RoomID	VARCHAR	5	PK	No		Unique room identifier
RoomName	VARCHAR	50		No	Main Hall, Yoga Hall,	Name/type of the room

					Small Dance Room	
MaxCapacity	INT	3		No	Must be > 0	Maximum number of people the room can hold

Table 4: Class

Field Name	Data Type	Length	Index	Null?	Validation	Description
ClassCode	VARCHAR	8	PK	No		Unique identifier for each class
ClassName	VARCHAR	50		No		Name of the class (e.g. Yoga, Zumba)
StaffID	VARCHAR	8	FK	No		References FitnessStaff (StaffID)
RoomID	VARCHAR	5	FK	No		References Room (RoomID)
DayOfWeek	VARCHAR	10		No	Monday.. ...Sunday	Day the class runs
StartTime	VARCHAR	5		No	HH:MM format	Scheduled start time
MaxClassSize	INT	3		No	Must be > 0	Maximum number of members who can book

Table 5: FacilityBooking

Field Name	Data Type	Length	Index	Null?	Validation	Description
BookingID	VARCHAR	8	PK	No		Unique booking reference
MemberID	VARCHAR	8	FK	No		References Member (MemberID)
RoomID	VARCHAR	5	FK	No		References Room (RoomID)
BookingDate	DATE			No	Cannot be past date	Date of the booking

StartTime	VARCHAR	5		No	HH:MM format	Start of the booked time slot
EndTime	VARCHAR	5		No	Max 2 hrs. after StartTime	End of the booked time slot
ActivityDescription	VARCHAR	100		Yes		Activity planned (e.g. Volleyball, Basketball)

Table 6: ClassBooking

Field Name	Data Type	Length	Index	Null?	Validation	Description
CBookingID	VARCHAR	8	PK	No		Unique class booking reference
MemberID	VARCHAR	8	FK	No		References Member (MemberID)
ClassCode	VARCHAR	8	FK	No		References Class (ClassCode)
BookingDate	DATE			No		Date the booking was made
ExtraCharge	CHAR	1		No	Y or N	Whether an extra Rs 500 charge applies

Task 3: Implementation of the Database

This section documents the SQL code used to build the Aspire Fitness database in MySQL. Part A covers the CREATE TABLE statements used to create each table. Part B covers the INSERT statements used to add example data into each table so we can test that everything works correctly.

Part A – Create Tables

Create Table: Member

This creates the Member table with MemberID as the primary key.

```sql
CREATE DATABASE AspireFitness;
USE AspireFitness;

CREATE TABLE Member (
  MemberID VARCHAR(8) NOT NULL,
  FirstName VARCHAR(50) NOT NULL,
  Surname VARCHAR(50) NOT NULL,
  Address VARCHAR(200) NOT NULL,
  Telephone VARCHAR(15) NOT NULL,
  Email VARCHAR(100) NOT NULL,
  DateOfBirth DATE NOT NULL,
  MedicalConditions VARCHAR(500),
  MembershipStartDate DATE NOT NULL,
  PRIMARY KEY (MemberID)
);
```

1	13:50:01	CREATE DATABASE AspireFitness	1 row(s) affected	0.078 sec
2	13:50:05	USE AspireFitness	0 row(s) affected	0.016 sec
3	13:50:09	CREATE TABLE Member (MemberID VARCHAR(8) NOT NULL, FirstName VARCHAR(50) NOT NULL,...	0 row(s) affected	0.109 sec

Figure 2: Member table created

Create Table: FitnessStaff

This creates the FitnessStaff table with StaffID as the primary key.

```sql
CREATE TABLE FitnessStaff (
  StaffID VARCHAR(8) NOT NULL,
  FirstName VARCHAR(50) NOT NULL,
  Surname VARCHAR(50) NOT NULL,
  Role VARCHAR(30) NOT NULL,
  ContactPhone VARCHAR(15) NOT NULL,
  PRIMARY KEY (StaffID)
);
```

4 13:50:14 CREATE TABLE FitnessStaff (StaffID VARCHAR(8) NOT NULL, FirstName VARCHAR(50) NOT NULL,... 0 row(s) affected 0.109 sec

Figure 3: FitnessStaff table created

Create Table: Room

This creates the Room table with RoomID as the primary key.

```sql
CREATE TABLE Room (
  RoomID VARCHAR(5) NOT NULL,
  RoomName VARCHAR(50) NOT NULL,
  MaxCapacity INT NOT NULL,
  PRIMARY KEY (RoomID)
);
```

5 13:50:18 CREATE TABLE Room (RoomID VARCHAR(5) NOT NULL, RoomName VARCHAR(50) NOT NULL, ... 0 row(s) affected 0.109 sec

Figure 4: Room table created

Create Table: Class

This creates the Class table with ClassCode as the primary key. StaffID and RoomID are foreign keys linking to the FitnessStaff and Room tables.

```sql
CREATE TABLE Class (
  ClassCode VARCHAR(8) NOT NULL,
  ClassName VARCHAR(50) NOT NULL,
  StaffID VARCHAR(8) NOT NULL,
  RoomID VARCHAR(5) NOT NULL,
  DayOfWeek VARCHAR(10) NOT NULL,
  StartTime VARCHAR(5) NOT NULL,
  MaxClassSize INT NOT NULL,
  PRIMARY KEY (ClassCode),
  FOREIGN KEY (StaffID) REFERENCES FitnessStaff(StaffID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

6 13:50:23 CREATE TABLE Class (ClassCode VARCHAR(8) NOT NULL, ClassName VARCHAR(50) NOT NULL, ... 0 row(s) affected 0.093 sec

Figure 5: Class table created

Create Table: FacilityBooking

This creates the FacilityBooking table with BookingID as the primary key. MemberID and RoomID are foreign keys linking to the Member and Room tables.

```sql
CREATE TABLE FacilityBooking (
  BookingID VARCHAR(8) NOT NULL,
  MemberID VARCHAR(8) NOT NULL,
  RoomID VARCHAR(5) NOT NULL,
  BookingDate DATE NOT NULL,
  StartTime VARCHAR(5) NOT NULL,
  EndTime VARCHAR(5) NOT NULL,
  ActivityDescription VARCHAR(100),
  PRIMARY KEY (BookingID),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

7 13:50:27 CREATE TABLE FacilityBooking (BookingID VARCHAR(8) NOT NULL, MemberID VARCHAR(8) NOT ... 0 row(s) affected 0.125 sec

Figure 6: FacilityBooking table created

Create Table: ClassBooking

This creates the ClassBooking table with CBookingID as the primary key. MemberID and ClassCode are foreign keys linking to the Member and Class tables.

```sql
CREATE TABLE ClassBooking (
    CBookingID VARCHAR(8) NOT NULL,
    MemberID VARCHAR(8) NOT NULL,
    ClassCode VARCHAR(8) NOT NULL,
    BookingDate DATE NOT NULL,
    ExtraCharge CHAR(1) NOT NULL,
    PRIMARY KEY (CBookingID),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (ClassCode) REFERENCES Class(ClassCode)
);
```

8 13:50:32 CREATE TABLE ClassBooking (CBookingID VARCHAR(8) NOT NULL, MemberID VARCHAR(8) NOT ... 0 row(s) affected 0.140 sec

Figure 7: ClassBooking table created

Part B – Insert Data

Insert Data: Member

```sql
INSERT INTO Member VALUES
('110','Kripa','Parajuli','Dadhikot, Bhaktapur','9001234567','kripa.parajuli@email.com','2006-09-18',NULL,'2024-01-10'),
('120','Bibek','Bharati','Dolalghat, Kavrepalanchowk','97711234002','bibek.bharati@email.com','2006-12-22',NULL,'2024-02-01'),
('130','Soniya','Gurung','Balkot, Bhaktapur','97711234003','soniya.gurung@email.com','2007-11-02','Asthma','2023-09-15'),
('140','Aananya','Shrestha','Patan, Lalitpur','97711234004','aananya.shrestha@email.com','2000-03-30','Knee injury','2024-03-01');
```

MemberID	FirstName	Surname	Address	Telephone	Email	DateOfBirth	MedicalConditions	MembershipStartDate
110	Kripa	Parajuli	Dadhikot, Bhaktapur	9001234567	kripa.parajuli@email.com	2006-09-18	NULL	2024-01-10
120	Bibek	Bharati	Dolalghat, Kavrepalanchowk	97711234002	bibek.bharati@email.com	2006-12-22	NULL	2024-02-01
130	Soniya	Gurung	Balkot, Bhaktapur	97711234003	soniya.gurung@email.com	2007-11-02	Asthma	2023-09-15
140	Aananya	Shrestha	Patan, Lalitpur	97711234004	aananya.shrestha@email.com	2000-03-30	Knee injury	2024-03-01
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

9 13:50:40 INSERT INTO Member VALUES ('110','Kripa','Parajuli','Dadhikot, Bhaktapur','9001234567','kripa.parajuli@... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 8: Member table with inserted data

Insert Data: FitnessStaff

```sql
INSERT INTO FitnessStaff VALUES
('S01','Ramesh','Shrestha','Class Instructor','9841234501'),
('S02','Suman','Tamang','Personal Trainer','9841234502'),
('S03','Anita','Maharjan','Class Instructor','9841234503'),
('S04','Bikash','Karki','Manager','9841234504');
```

StaffID	FirstName	Surname	Role	ContactPhone
S01	Ramesh	Shrestha	Class Instructor	9841234501
S02	Suman	Tamang	Personal Trainer	9841234502
S03	Anita	Maharjan	Class Instructor	9841234503
S04	Bikash	Karki	Manager	9841234504
NULL	NULL	NULL	NULL	NULL

10 13:50:44 INSERT INTO FitnessStaff VALUES ('S01','Ramesh','Shrestha','Class Instructor','9841234501'), ('S02','Sum... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.078 sec

Figure 9: FitnessStaff table with inserted data

Insert Data: Room

```sql
INSERT INTO Room VALUES
('R001','Main Hall',80),
('R002','Yoga Hall',25),
('R003','Small Dance Room',10);
```

RoomID	RoomName	MaxCapacity
R001	Main Hall	80
R002	Yoga Hall	25
R003	Small Dance Room	10
NULL	NULL	NULL

12 13:50:53 INSERT INTO Class VALUES ('CLS001','Yoga Flow','S01','R002','Monday','09:00',20), ('CLS002','Zumba','... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 10: Room table with inserted data

Insert Data: Class

```
INSERT INTO Class VALUES
('CLS001','Yoga Flow','S01','R002','Monday','09:00',20),
('CLS002','Zumba','S03','R001','Wednesday','18:30',40),
('CLS003','Pilates','S01','R003','Friday','10:00',10),
('CLS004','Tai Chi','S03','R002','Tuesday','08:00',15);
```

ClassCode	ClassName	StaffID	RoomID	DayOfWeek	StartTime	MaxClassSize
CLS001	Yoga Flow	S01	R002	Monday	09:00	20
CLS002	Zumba	S03	R001	Wednesday	18:30	40
CLS003	Pilates	S01	R003	Friday	10:00	10
CLS004	Tai Chi	S03	R002	Tuesday	08:00	15
NULL	NULL	NULL	NULL	NULL	NULL	NULL

12 13:50:53 INSERT INTO Class VALUES ('CLS001','Yoga Flow','S01','R002','Monday','09:00',20), ('CLS002','Zumba','... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 11: Class table with inserted data

Insert Data: FacilityBooking

```
INSERT INTO FacilityBooking VALUES
('FB001','110','R001','2026-03-10','10:00','12:00','Volleyball'),
('FB002','120','R002','2026-03-11','14:00','16:00','Yoga practice'),
('FB003','130','R001','2026-03-12','09:00','11:00','Basketball'),
('FB004','140','R003','2026-03-13','13:00','15:00','Dance practice');
```

BookingID	MemberID	RoomID	BookingDate	StartTime	EndTime	ActivityDescription
FB001	110	R001	2026-03-10	10:00	12:00	Volleyball
FB002	120	R002	2026-03-11	14:00	16:00	Yoga practice
FB003	130	R001	2026-03-12	09:00	11:00	Basketball
FB004	140	R003	2026-03-13	13:00	15:00	Dance practice
NULL	NULL	NULL	NULL	NULL	NULL	NULL

13 13:50:57 INSERT INTO FacilityBooking VALUES ('FB001','110','R001','2026-03-10','10:00','12:00','Volleyball'), ('FB0... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.062 sec

Figure 12: FacilityBooking table with inserted data

Insert Data: ClassBooking

```sql
INSERT INTO ClassBooking VALUES
('CB001','110','CLS001','2026-03-05','N'),
('CB002','110','CLS002','2026-03-05','N'),
('CB003','120','CLS003','2026-03-06','N'),
('CB004','130','CLS001','2026-03-06','N'),
('CB005','130','CLS004','2026-03-07','N'),
('CB006','130','CLS002','2026-03-07','Y'),
('CB007','130','CLS001','2026-03-08','N'),
('CB008','130','CLS002','2026-03-08','N'),
('CB009','130','CLS003','2026-03-09','Y');
```

CBookingID	MemberID	ClassCode	BookingDate	ExtraCharge
CB001	110	CLS001	2026-03-05	N
CB002	110	CLS002	2026-03-05	N
CB003	120	CLS003	2026-03-06	N
CB004	130	CLS001	2026-03-06	N
CB005	130	CLS004	2026-03-07	N
CB006	130	CLS002	2026-03-07	Y
CB007	130	CLS001	2026-03-08	N
CB008	130	CLS002	2026-03-08	N
CB009	130	CLS003	2026-03-09	Y
NULL	NULL	NULL	NULL	NULL

14 13:51:01 INSERT INTO ClassBooking VALUES ('CB001','110','CLS001','2026-03-05','N'), ('CB002','110','CLS002','2... 9 row(s) affected Records: 9 Duplicates: 0 Warnings: 0 0.063 sec

Figure 13: ClassBooking table with inserted data

Task 4: Query Design and Implementation

This section covers the SQL queries written for Aspire Fitness. A query is used to find and display useful information from the database. Part A lists all the queries and explains what each one does. Part B shows the full SQL code for each query along with a simple explanation of how it works.

Part A – List of Queries and Their Purpose

#	Query Name	Justification
1	Full Member List	Shows a list of all registered members. Useful for admin tasks like sending reminders or newsletters.
2	Current Week Facility Bookings	Shows all room bookings for the current week. Helps staff manage room usage and scheduling.
3	Classes by a Specific Instructor	Lists all classes taught by a specific instructor. Useful for checking staff workload and timetables.
4	Classes in a Specific Room	Shows all classes taking place in a particular room. Helps identify which time slots are free.
5	Member Booking Summary	Shows all room and class bookings for a specific member in the current week. Helps staff check what a member has booked.
6	Class Bookings Count per Member	Counts how many classes each member has booked this week. Helps staff see who is close to or over their free class limit.
7	Most Active Members This Month	Shows which members have made the most bookings this month. Useful for identifying the most engaged members.
8	Members Over Free Class Limit	Shows members who have booked more than five classes this week. These members need to be charged Rs 500 for each extra class.
9	Room Usage Summary	Counts how many bookings each room has received. Helps management see which rooms are used the most.
10	Members with Extra Charges	Lists members who have extra class charges this week. Useful for the billing and payment process.

Part B – SQL Queries Implemented

Query 1: Full Member List

Detail	Description
Purpose	Shows a list of all registered members ordered by surname.
Explanation	SELECT is used to pick the columns we want to show from the Member table. ORDER BY Surname sorts the results from A to Z by surname.

```
SELECT MemberID, FirstName, Surname, Telephone, Email
FROM Member
ORDER BY Surname;
```

MemberID	FirstName	Surname	Telephone	Email
120	Bibek	Bharati	97711234002	bibek.bharati@email.com
130	Soniya	Gurung	97711234003	soniya.gurung@email.com
110	Kripa	Parajuli	9001234567	kripa.parajuli@email.com
140	Aananya	Shrestha	97711234004	aananya.shrestha@email.com
NULL	NULL	NULL	NULL	NULL

Figure 14: Query 1 output

Query 2: Current Week Facility Bookings

Detail	Description
Purpose	Shows all room bookings made in the current week.
Explanation	JOIN connects the FacilityBooking, Member and Room tables so we can see the member's name and room name together. WHERE filters the results to only show bookings from the current week.

```
SELECT FacilityBooking.BookingID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Room.RoomName,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       FacilityBooking.EndTime,
       FacilityBooking.ActivityDescription
FROM FacilityBooking
JOIN Member ON FacilityBooking.MemberID = Member.MemberID
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE MONTH(FacilityBooking.BookingDate) = MONTH(CURDATE())
AND YEAR(FacilityBooking.BookingDate) = YEAR(CURDATE())
ORDER BY FacilityBooking.BookingDate, FacilityBooking.StartTime;
```

	BookingID	MemberName	RoomName	BookingDate	StartTime	EndTime	ActivityDescription
▶	FB001	Kripa Parajuli	Main Hall	2026-03-10	10:00	12:00	Volleyball
	FB002	Bibek Bharati	Yoga Hall	2026-03-11	14:00	16:00	Yoga practice
	FB003	Soniya Gurung	Main Hall	2026-03-12	09:00	11:00	Basketball
	FB004	Aananya Shrestha	Small Dance Room	2026-03-13	13:00	15:00	Dance practice

Figure 15: Query 2 output

Query 3: Classes by a Specific Instructor

Detail	Description
Purpose	Shows all classes taught by a specific instructor.
Explanation	JOIN connects the Class, FitnessStaff and Room tables. WHERE filters by the instructor's StaffID. To search for a different instructor, just change the StaffID value in the WHERE clause.

```
SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       Room.RoomName,
       Class.MaxClassSize,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Class.StaffID = 'S01'
ORDER BY Class.DayOfWeek, Class.StartTime;
```

	ClassCode	ClassName	DayOfWeek	StartTime	RoomName	MaxClassSize	InstructorName
▶	CLS003	Pilates	Friday	10:00	Small Dance Room	10	Ramesh Shrestha
	CLS001	Yoga Flow	Monday	09:00	Yoga Hall	20	Ramesh Shrestha

Figure 16: Query 3 output

Query 4: Classes in a Specific Room

Detail	Description
Purpose	Shows all classes scheduled in a specific room.

Explanation	JOIN connects the Class, FitnessStaff and Room tables. WHERE filters by RoomID. ORDER BY sorts the results by day and time so the timetable is easy to read.

```sql
SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Room.RoomID = 'R002'
ORDER BY Class.DayOfWeek, Class.StartTime;
```

ClassCode	ClassName	DayOfWeek	StartTime	InstructorName
CLS001	Yoga Flow	Monday	09:00	Ramesh Shrestha
CLS004	Tai Chi	Tuesday	08:00	Anita Maharjan

Figure 17: Query 4 output

Query 5: Member Booking Summary

Detail	Description
Purpose	Shows all room and class bookings for a specific member in the current week.
Explanation	UNION ALL combines results from FacilityBooking and ClassBooking into one list. A label column called BookingType is added so we can tell which type of booking each row is. WHERE filters by member ID and current week.

```sql
SELECT 'Facility Booking' AS BookingType,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       Room.RoomName,
       FacilityBooking.ActivityDescription AS Detail
FROM FacilityBooking
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE FacilityBooking.MemberID = '110'
UNION ALL
SELECT 'Class Booking' AS BookingType,
       ClassBooking.BookingDate,
       Class.StartTime,
       Room.RoomName,
       Class.ClassName AS Detail
FROM ClassBooking
JOIN Class ON ClassBooking.ClassCode = Class.ClassCode
JOIN Room ON Class.RoomID = Room.RoomID
WHERE ClassBooking.MemberID = '110'
ORDER BY BookingDate, StartTime;
```

BookingType	BookingDate	StartTime	RoomName	Detail
Class Booking	2026-03-05	09:00	Yoga Hall	Yoga Flow
Class Booking	2026-03-05	18:30	Main Hall	Zumba
Facility Booking	2026-03-10	10:00	Main Hall	Volleyball

Figure 18: Query 5 output

Query 6: Class Bookings Count per Member

Detail	Description
Purpose	Counts how many classes each member has booked this week.
Explanation	GROUP BY groups the results by member so we can count their bookings separately. COUNT counts the number of bookings for each member. ORDER BY sorts so the member with the most bookings appears at the top.

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
ORDER BY TotalClassBookings DESC;
```

MemberID	MemberName	TotalClassBookings
130	Soniya Gurung	6
110	Kripa Parajuli	2
120	Bibek Bharati	1

Figure 19: Query 6 output

Query 7: Most Active Members This Month

Detail	Description
Purpose	Shows which members have made the most bookings this month by adding up their room and class bookings.
Explanation	Two subqueries count room bookings and class bookings separately for each member. LEFT JOIN combines both counts into one result. IFNULL makes sure that if a member has no bookings, it shows 0 instead of blank. Results are sorted by total activity from highest to lowest.

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       IFNULL(FacilityCount.Total, 0) AS FacilityBookings,
       IFNULL(ClassCount.Total, 0) AS ClassBookings,
       IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) AS TotalActivity
FROM Member
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM FacilityBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) FacilityCount ON Member.MemberID = FacilityCount.MemberID
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM ClassBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) ClassCount ON Member.MemberID = ClassCount.MemberID
WHERE IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) > 0
ORDER BY TotalActivity DESC;
```

MemberID	MemberName	FacilityBookings	ClassBookings	TotalActivity
130	Soniya Gurung	1	6	7
110	Kripa Parajuli	1	2	3
120	Bibek Bharati	1	1	2
140	Aananya Shrestha	1	0	1

Figure 20: Query 7 output

Query 8: Members Over Free Class Limit

Detail	Description
Purpose	Shows members who have booked more than five classes this week and calculates their extra charge.
Explanation	GROUP BY groups the bookings by member. HAVING filters to only show members with more than five class bookings. The extra charge is calculated by multiplying the number of extra classes by 500.

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings,
       (COUNT(*) - 5) * 500 AS ExtraChargeRs
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
HAVING COUNT(*) > 5
ORDER BY TotalClassBookings DESC;
```

	MemberID	MemberName	TotalClassBookings	ExtraChargeRs
▶	130	Soniya Gurung	6	500

Figure 21: Query 8 output

Query 9: Room Usage Summary

Detail	Description
Purpose	Shows how many bookings each room has received in total.
Explanation	LEFT JOIN connects Room and FacilityBooking so even rooms with no bookings still appear. GROUP BY groups by room. COUNT counts the bookings for each room. Results are sorted from most booked to least booked.

```sql
SELECT Room.RoomID,
       Room.RoomName,
       Room.MaxCapacity,
       COUNT(FacilityBooking.BookingID) AS TotalBookings
FROM Room
LEFT JOIN FacilityBooking ON Room.RoomID = FacilityBooking.RoomID
GROUP BY Room.RoomID, Room.RoomName, Room.MaxCapacity
ORDER BY TotalBookings DESC;
```

	RoomID	RoomName	MaxCapacity	TotalBookings
▶	R001	Main Hall	80	2
	R002	Yoga Hall	25	1
	R003	Small Dance Room	10	1

Figure 22: Query 9 output

Query 10: Members with Extra Charges

Detail	Description
Purpose	Shows all members who have at least one extra charge class booking this week.
Explanation	WHERE filters records where ExtraCharge is Y and the booking is within the current week. JOIN connects to the Member table to show the member's name. DISTINCT makes sure each member only appears once even if they have multiple extra charge bookings.

```
SELECT DISTINCT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Member.Email
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
WHERE ClassBooking.ExtraCharge = 'Y'
ORDER BY Member.Surname;
```

	MemberID	Surname	MemberName	Email
▶	130	Gurung	Soniya Gurung	soniya.gurung@email.com

Figure 23: Query 10 output

Discussion and Critical Reflection

Overall, this project went well and I completed all four tasks successfully. One challenge I faced was deciding how to split the data into tables. I created two separate booking tables because room bookings and class bookings store different information. Putting them together would have left many columns empty which is poor database design.

I also learned about normalization, which means organizing data so it is not repeated. For example, keeping the Class table separate means if an instructor changes, I only update one record instead of many.

Writing the queries taught me the difference between WHERE and HAVING. WHERE filters before grouping and HAVING filters after grouping. Planning the structure before writing any code also saved a lot of time during implementation.

In the future I would add a payments table to track weekly charges and use database triggers to automatically enforce booking rules.

Conclusion

This report has documented the full process of designing and implementing a database for Aspire Fitness. I followed each stage carefully from analyzing the scenario and drawing the ERD to building the tables in MySQL and writing ten SQL queries to prove the system works. Through this assignment I improved my understanding of database design, normalization, and SQL which has given me a strong foundation for future learning.

References

Oracle Corporation (2024) MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/ [Accessed: March 2026].

Connolly, T. and Begg, C. (2014) Database Systems: A Practical Approach to Design, Implementation, and Management. 6th edn. Harlow: Pearson Education.

Date, C.J. (2003) An Introduction to Database Systems. 8th edn. Boston: Addison-Wesley.

Appendix

```sql
CREATE DATABASE AspireFitness;
USE AspireFitness;

CREATE TABLE Member (
    MemberID VARCHAR(8) NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    Surname VARCHAR(50) NOT NULL,
    Address VARCHAR(200) NOT NULL,
    Telephone VARCHAR(15) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    DateOfBirth DATE NOT NULL,
    MedicalConditions VARCHAR(500),
    MembershipStartDate DATE NOT NULL,
    PRIMARY KEY (MemberID)
);

CREATE TABLE FitnessStaff (
    StaffID VARCHAR(8) NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    Surname VARCHAR(50) NOT NULL,
    Role VARCHAR(30) NOT NULL,
    ContactPhone VARCHAR(15) NOT NULL,
    PRIMARY KEY (StaffID)
);
```

```sql

CREATE TABLE Room (
    RoomID VARCHAR(5) NOT NULL,
    RoomName VARCHAR(50) NOT NULL,
    MaxCapacity INT NOT NULL,
    PRIMARY KEY (RoomID)
);

CREATE TABLE Class (
    ClassCode VARCHAR(8) NOT NULL,
    ClassName VARCHAR(50) NOT NULL,
    StaffID VARCHAR(8) NOT NULL,
    RoomID VARCHAR(5) NOT NULL,
    DayOfWeek VARCHAR(10) NOT NULL,
    StartTime VARCHAR(5) NOT NULL,
    MaxClassSize INT NOT NULL,
    PRIMARY KEY (ClassCode),
    FOREIGN KEY (StaffID) REFERENCES FitnessStaff(StaffID),
    FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

```sql
CREATE TABLE FacilityBooking (
  BookingID VARCHAR(8) NOT NULL,
  MemberID VARCHAR(8) NOT NULL,
  RoomID VARCHAR(5) NOT NULL,
  BookingDate DATE NOT NULL,
  StartTime VARCHAR(5) NOT NULL,
  EndTime VARCHAR(5) NOT NULL,
  ActivityDescription VARCHAR(100),
  PRIMARY KEY (BookingID),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);

CREATE TABLE ClassBooking (
  CBookingID VARCHAR(8) NOT NULL,
  MemberID VARCHAR(8) NOT NULL,
  ClassCode VARCHAR(8) NOT NULL,
  BookingDate DATE NOT NULL,
  ExtraCharge CHAR(1) NOT NULL,
  PRIMARY KEY (CBookingID),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (ClassCode) REFERENCES Class(ClassCode)
);
```

```sql
INSERT INTO Member VALUES
('110','Kripa','Parajuli','Dadhikot, Bhaktapur','9001234567','kripa.parajuli@email.com','2006-09-18',NULL,'2024-01-10'),
('120','Bibek','Bharati','Dolalghat, Kavrepalanchowk','97711234002','bibek.bharati@email.com','2006-12-22',NULL,'2024-02-01'),
('130','Soniya','Gurung','Balkot, Bhaktapur','97711234003','soniya.gurung@email.com','2007-11-02','Asthma','2023-09-15'),
('140','Aananya','Shrestha','Patan, Lalitpur','97711234004','aananya.shrestha@email.com','2000-03-30','Knee injury','2024-03-01');

INSERT INTO FitnessStaff VALUES
('S01','Ramesh','Shrestha','Class Instructor','9841234501'),
('S02','Suman','Tamang','Personal Trainer','9841234502'),
('S03','Anita','Maharjan','Class Instructor','9841234503'),
('S04','Bikash','Karki','Manager','9841234504');

INSERT INTO Room VALUES
('R001','Main Hall',80),
('R002','Yoga Hall',25),
('R003','Small Dance Room',10);

INSERT INTO Class VALUES
('CLS001','Yoga Flow','S01','R002','Monday','09:00',20),
('CLS002','Zumba','S03','R001','Wednesday','18:30',40),
('CLS003','Pilates','S01','R003','Friday','10:00',10),
('CLS004','Tai Chi','S03','R002','Tuesday','08:00',15);
```

```sql
INSERT INTO FacilityBooking VALUES
('FB001','110','R001','2026-03-10','10:00','12:00','Volleyball'),
('FB002','120','R002','2026-03-11','14:00','16:00','Yoga practice'),
('FB003','130','R001','2026-03-12','09:00','11:00','Basketball'),
('FB004','140','R003','2026-03-13','13:00','15:00','Dance practice');

INSERT INTO ClassBooking VALUES
('CB001','110','CLS001','2026-03-05','N'),
('CB002','110','CLS002','2026-03-05','N'),
('CB003','120','CLS003','2026-03-06','N'),
('CB004','130','CLS001','2026-03-06','N'),
('CB005','130','CLS004','2026-03-07','N'),
('CB006','130','CLS002','2026-03-07','Y'),
('CB007','130','CLS001','2026-03-08','N'),
('CB008','130','CLS002','2026-03-08','N'),
('CB009','130','CLS003','2026-03-09','Y');


SELECT MemberID, FirstName, Surname, Telephone, Email
FROM Member
ORDER BY Surname;

```

```sql

SELECT FacilityBooking.BookingID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Room.RoomName,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       FacilityBooking.EndTime,
       FacilityBooking.ActivityDescription
FROM FacilityBooking
JOIN Member ON FacilityBooking.MemberID = Member.MemberID
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE MONTH(FacilityBooking.BookingDate) = MONTH(CURDATE())
AND YEAR(FacilityBooking.BookingDate) = YEAR(CURDATE())
ORDER BY FacilityBooking.BookingDate, FacilityBooking.StartTime;


```

```sql
SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       Room.RoomName,
       Class.MaxClassSize,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Class.StaffID = 'S01'
ORDER BY Class.DayOfWeek, Class.StartTime;


SELECT Class.ClassCode,
       Class.ClassName,
       Class.DayOfWeek,
       Class.StartTime,
       CONCAT(FitnessStaff.FirstName,' ',FitnessStaff.Surname) AS InstructorName
FROM Class
JOIN FitnessStaff ON Class.StaffID = FitnessStaff.StaffID
JOIN Room ON Class.RoomID = Room.RoomID
WHERE Room.RoomID = 'R002'
ORDER BY Class.DayOfWeek, Class.StartTime;
```

```sql
SELECT 'Facility Booking' AS BookingType,
       FacilityBooking.BookingDate,
       FacilityBooking.StartTime,
       Room.RoomName,
       FacilityBooking.ActivityDescription AS Detail
FROM FacilityBooking
JOIN Room ON FacilityBooking.RoomID = Room.RoomID
WHERE FacilityBooking.MemberID = '110'
UNION ALL
SELECT 'Class Booking' AS BookingType,
       ClassBooking.BookingDate,
       Class.StartTime,
       Room.RoomName,
       Class.ClassName AS Detail
FROM ClassBooking
JOIN Class ON ClassBooking.ClassCode = Class.ClassCode
JOIN Room ON Class.RoomID = Room.RoomID
WHERE ClassBooking.MemberID = '110'
ORDER BY BookingDate, StartTime;
```

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
ORDER BY TotalClassBookings DESC;
```

```sql
SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       IFNULL(FacilityCount.Total, 0) AS FacilityBookings,
       IFNULL(ClassCount.Total, 0) AS ClassBookings,
       IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) AS TotalActivity
FROM Member
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM FacilityBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) FacilityCount ON Member.MemberID = FacilityCount.MemberID
LEFT JOIN (
    SELECT MemberID, COUNT(*) AS Total
    FROM ClassBooking
    WHERE MONTH(BookingDate) = MONTH(CURDATE())
    AND YEAR(BookingDate) = YEAR(CURDATE())
    GROUP BY MemberID
) ClassCount ON Member.MemberID = ClassCount.MemberID
WHERE IFNULL(FacilityCount.Total, 0) + IFNULL(ClassCount.Total, 0) > 0
ORDER BY TotalActivity DESC;
```

fitness*

Limit to 1000 rows

```sql

SELECT Member.MemberID,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       COUNT(*) AS TotalClassBookings,
       (COUNT(*) - 5) * 500 AS ExtraChargeRs
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
GROUP BY Member.MemberID, Member.FirstName, Member.Surname
HAVING COUNT(*) > 5
ORDER BY TotalClassBookings DESC;


SELECT Room.RoomID,
       Room.RoomName,
       Room.MaxCapacity,
       COUNT(FacilityBooking.BookingID) AS TotalBookings
FROM Room
LEFT JOIN FacilityBooking ON Room.RoomID = FacilityBooking.RoomID
GROUP BY Room.RoomID, Room.RoomName, Room.MaxCapacity
ORDER BY TotalBookings DESC;

```

fitness*

Limit to 1000 rows

```sql

SELECT DISTINCT Member.MemberID,
       Member.Surname,
       CONCAT(Member.FirstName,' ',Member.Surname) AS MemberName,
       Member.Email
FROM ClassBooking
JOIN Member ON ClassBooking.MemberID = Member.MemberID
WHERE ClassBooking.ExtraCharge = 'Y'
ORDER BY Member.Surname;


```